Inferring network from bulk data with CausFate

Lan-lab

1/14/2026

Structure learning

Load packages

```
library(Seurat)
library(foreach)
library(tidyverse)
library(bnlearn)
library(Matrix)
library(space)
library(infotheo)

library(causfate)
```

Load data

```
data("HMMA")
head(HMMA)
```

```
##                     HSC  MPPa     MPPb     sCMP   MEP      GMP      MkP   pCFU-E
## 0610005C13Rik 4.400000 4.420 4.170000 4.180000 4.250 4.370000 4.890000 4.320000
## 0610006L08Rik 3.810000 3.950 4.030000 3.960000 4.390 4.460000 4.850000 4.540000
## 0610007C21Rik 8.950000 6.340 7.650000 6.550000 8.040 8.630000 7.030000 7.390000
## 0610007L01Rik 7.693333 8.320 8.423333 8.446667 9.300 9.876667 7.453333 8.023333
## 0610007P08Rik 5.430000 5.742 6.088000 5.870000 6.166 5.790000 5.496000 5.682000
## 0610007P14Rik 7.930000 7.700 7.760000 7.160000 8.820 8.820000 7.530000 7.540000
##                CLP      BLP  DN1
## 0610005C13Rik 4.15 4.510000 4.19
## 0610006L08Rik 4.38 4.330000 4.28
## 0610007C21Rik 9.03 8.740000 8.68
## 0610007L01Rik 8.35 8.213333 7.69
## 0610007P08Rik 5.89 5.896000 5.63
## 0610007P14Rik 8.47 8.280000 7.85
```

```
dim(HMMA)
```

```
## [1] 21678    11
```

Extract key genes

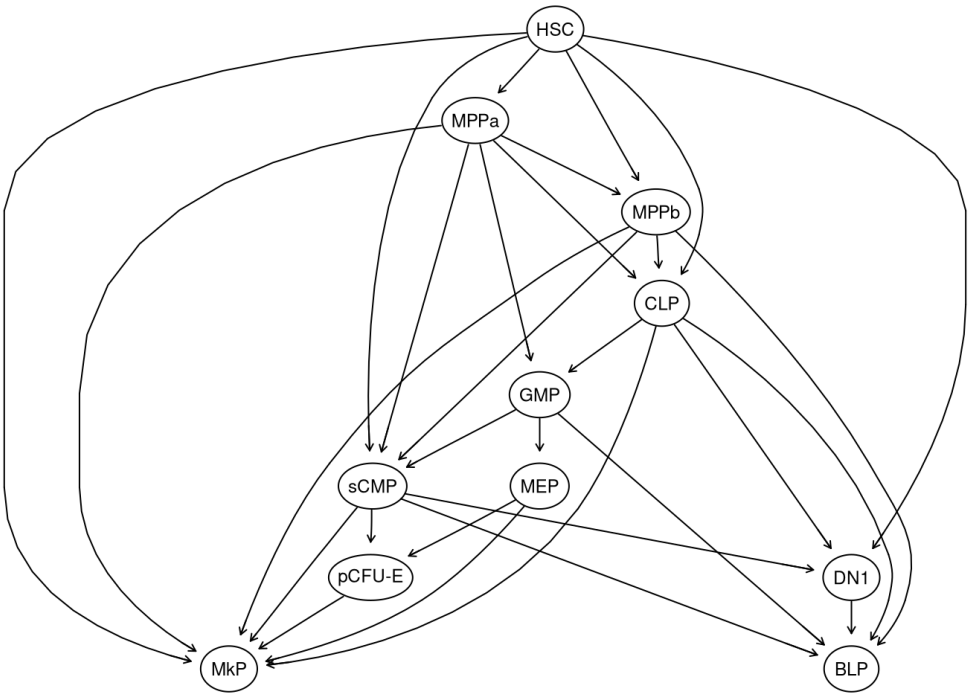
To accelerate the process, we only use the top 3000 genes for network learning. Here, we use `infotheo::mutinformation` to determine key gene markers.

```
ctypes <- colnames(HMMA)
# Binning expression values
HMMA_disc <- infotheo::discretize(HMMA) %>% as.matrix()

# Calculate the mutual information
doParallel::registerDoParallel(cores = 5)
HMMA_glist <- foreach(i = 1:nrow(HMMA), .combine = c) %dopar% {
  infotheo::mutinformation(HMMA_disc[i, ], ctypes)
}
names(HMMA_glist) <- rownames(HMMA)
HMMA_glist <- sort(HMMA_glist, decreasing = T)
```

Structure learning

The Bayesian network structure is learned with `BNLearning`.

```
dat <- HMMA[names(HMMA_glist)[1:3000], ]
struc_smpls <- BNLearning(dat,
  params = seq(from = 0.08, to = 0.23, length.out = 10),
  root = "HSC", mode = "bulk", ncores = 5, ugMethod = "cmi2ni", dagMethod = "hc"
)
net_struc <- combineDAGs(struc_smpls, Emin = 20, Emax = 30)
net_struc <- rmCyc(net_struc)
e <- df2bn(net_struc, ctypes, plot = TRUE)

bnlearn::graphviz.plot(e, shape = "ellipse")
```



Structure refining

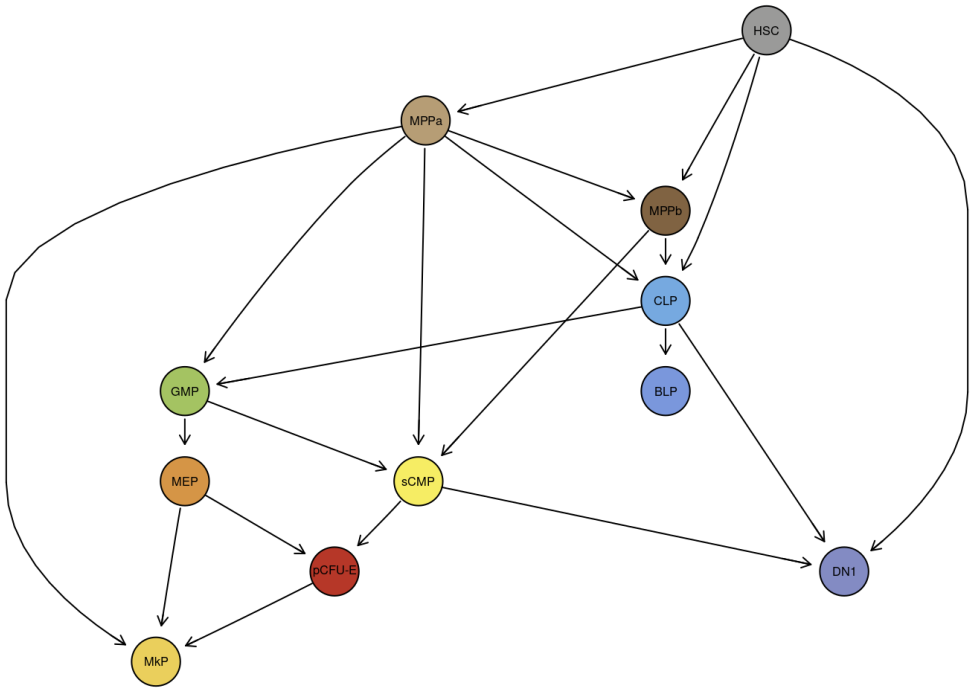
In the following step, we make use of gene variability information to prune our graph with `trimDAG`.

`trimDAG` automatically plots the modified structure for you. This feature can be turned off by setting `plot = FALSE`.

```
outputDAG <- trimDAG(HMMA, e, 2, 3, .9, plot = FALSE)

# Code for coloring is omitted for brevity.
```

```
library(igraph)
library(graph)
library(Rgraphviz)
node_col <- c(
  "HSC" = "#9A9A99",
  "MPPa" = "#B69D75",
  "MPPb" = "#806342",
  "CLP" = "#76A9E0",
  "GMP" = "#A4C362",
  "MEP" = "#D59546",
  "Ery" = "#D88176",
  "MkP" = "#EACF5C",
  "DN1" = "#838BC3",
  "sCMP" = "#F6ED64",
  "BLP" = "#7A97DC"
)
edge_col <- c(`HSC~MPPa` = "black", `HSC~MPPb` = "black", `HSC~sCMP` = "gray",
              `HSC~CLP` = "gray", `HSC~DN1` = "gray", `MPPa~MPPb` = "black",
              `MPPa~sCMP` = "black", `MPPa~GMP` = "gray", `MPPa~MkP` = "gray",
```

```
                `MPPa~CLP` = "black", `MPPb~sCMP` = "black", `MPPb~CLP` = "black",
                `sCMP~GMP` = "black", `sCMP~Ery` = "gray", `sCMP~DN1` = "red",
                `MEP~MkP` = "black", `MEP~Ery` = "black", `GMP~MEP` = "red",
                `Ery~MkP` = "red", `CLP~GMP` = "red", `CLP~BLP` = "black",
                `CLP~DN1` = "black")
tmp <- graphviz.plot(outputDAG, shape = "circle", render = F)
nodeRenderInfo(tmp)$fill <- node_col
edgeRenderInfo(tmp)$col <- edge_col
renderGraph(tmp)
```



Calculating Effect Matrix

Having learnt the network structure, following functions are used to extract causal information of genes.

1. `PerturbResult`: calculates the BN coefficients after gene perturbation.
2. `EffectMatrix`: extracts gene contribution to each edge in the structure.
3. `diffScore`: calculates gene contribution to a subset of edges.
4. `dsRank`: gives ranked gene list according to previous scores.

```
perturbRes <- PerturbResult(outputDAG, HMMA, mode = "bulk")
effMat <- EffectMatrix(perturbRes, mode = "mean")

edgeSet <- setEdges(fromSet = ctypes, toSet = ctypes) # Calculate for all edges
effectScore <- diffScore(effMat, edgeSet)
geneList <- dsRank(effectScore)
head(geneList)
```

```
## [1] "Gm10883 or Gm1420 or Gm7202 or Igk-C or Igk-J1 or Igk-V28"
## [2] "Car1"
## [3] "Pf4"
## [4] "Mpo"
## [5] "Gm10482"
## [6] "Aldh1a1"
```